

Comparison Between LangGraph and LangChain Approaches

In this project, we implemented a structured complaint-handling workflow using LangGraph and compared it conceptually with a more traditional LangChain approach. Although both tools are built for working with large language models, they are designed with different goals in mind and lead to different architectural choices.

LangGraph is centered around explicit workflow design. It allows the developer to define nodes, edges, state transitions, and execution paths in a deterministic way. In our implementation, each complaint moved through clearly defined steps: intake, validation, investigation, resolution, and closure. The state was tracked at every stage, and the workflow path was stored and printed at the end. This made the execution traceable and predictable. If something went wrong, it was easy to see at which step the issue occurred.

LangChain, on the other hand, is generally more flexible and LLM-driven. Instead of defining a strict graph structure, developers often create chains where the model handles more of the reasoning dynamically. This can reduce boilerplate and make it faster to prototype ideas. However, it also means that the control flow is less explicit. In complex systems, this may make debugging harder because the reasoning is mostly embedded inside the model's responses rather than in a visible workflow structure.

One major difference is determinism. LangGraph enforces a defined path. Even if the LLM is used inside nodes, the overall flow is controlled by the graph. In contrast, LangChain chains often rely on the LLM to decide what to do next, especially in agent-based setups. This gives more flexibility but less structural control.

Another important difference is state management. With LangGraph, the state is clearly typed (in our case using a TypedDict) and passed from node to node. Each function updates the state explicitly. This makes the system easier to scale and maintain because changes are localized and predictable. In a simpler LangChain setup, state is often passed implicitly through prompts or memory objects, which can become harder to track as the application grows.

In terms of use cases, LangGraph is better suited for structured workflows, production systems, and applications that require traceability and reliability. Examples include customer service automation, internal business processes, or compliance-related flows where each step must be documented. The clarity of transitions and the ability to visualize the workflow path are strong advantages.

LangChain is more appropriate for creative or exploratory tasks. For example, open-ended reasoning, conversational agents, brainstorming tools, or tasks where the flow should adapt dynamically to user input. It allows rapid development and experimentation without designing a full graph structure.

There are trade-offs. LangGraph requires more upfront design effort. The developer must think carefully about nodes and transitions. This adds complexity at the beginning but pays off in stability and maintainability. LangChain can be faster to set up, but as complexity increases, the lack of explicit structure can make the system harder to debug or extend.

In conclusion, both approaches are valuable but serve different purposes. LangGraph is preferable when building structured, multi-step workflows that need transparency and control. LangChain is better for flexible, creative, and less predictable tasks. The choice depends on the level of structure required and the long-term goals of the application.